

PORTFOLIO PROJECT

CyberAudit & Solutions

Plateforme de gestion d'audit pour PME
Stage 3: Technical Documentation

Holberton School Dijon | 2026

Tommy JOUHANS : Développeur Front-End
James ROUSSEL : Développeur Back-End



Sommaire

1. Introduction : page 3

- 1.1 Contexte : page 3
- 1.2 Portée du document : page 3
- 1.3 Public cible : page 3
- 1.4 Stack technique : page 3

2. User Stories et Maquettes : page 4

- 2.1 Personas : page 4
- 2.2 User Stories Priorisées : page 4
- 2.3 Description des écrans (11 maquettes) : page 5

3. Architecture du Système : page 16

- 3.1 Vue d'ensemble : Architecture en couches : page 16
- 3.2 Flux de données : Demande d'audit : page 17
- 3.3 Choix architecturaux justifiés : page 17

4. Composants, Classes et Base de données : page 18

- 4.1 Diagramme de la base de données : page 18
- 4.2 Diagramme des classes du backend : page 20
- 4.3 Composants frontend (React + Tailwind) : page 23
- 4.4 Génération de rapports PDF : page 29
- 4.5 Supervision et observabilité : page 30
- 4.6 Migrations et remplissage de la base de données : page 31

5. Diagrammes de séquence : page 34

- 5.1 Enregistrement + Connexion JWT : page 34
- 5.2 Soumission de la demande d'audit (asynchrone) : page 36
- 5.3 Génération de rapports PDF (admin) : page 38

6. Documentation de l'API : page 40

- 6.1 APIs externes consommées : page 40
- 6.2 API interne : Point de terminaison REST : page 41
- 6.3 Conventions Transversales : page 41

7. Stratégie SCM & QA : page 43

- 7.1 Gestion du Code Source (Git) : page 43
- 7.2 Stratégie QA : page 43
- 7.3 Tests de Sécurité Manuels : page 44
- 7.4 Pipeline CI/CD : page 44

8. Annexes : page 46

- 8.1 Glossaire : page 46
- 8.2 Cartographie Étape 3 ↔ RNCP 5 : page 46
- 8.3 Sources & Références Techniques : page 47
- 8.4 Liens Internes du Projet : page 47

Introduction

1.1 Contexte

CyberAudit & Solutions est une plateforme web full-stack qui met en relation les PME et TPE françaises ne disposant pas d'équipe informatique interne avec des experts en cybersécurité. Selon l'ANSSI 2023, 45 % des PME n'ont pas d'équipe IT dédiée et restent exposées aux cyberattaques (rançongiciels, hameçonnage, fuites de données).

1.2 Périmètre du document

Cette documentation technique couvre la conception détaillée du MVP.
Elle décrit :

- Les besoins utilisateurs formalisés en User Stories priorisées.
- L'architecture logicielle et le flux de données ; le modèle de données (PostgreSQL ou MySQL relationnel).
- Les classes back-end, les composants front-end, les scénarios d'interaction, ainsi que les APIs exposées et consommées.
- La stratégie de gestion du code et de la qualité.

1.3 Public cible

- **Équipe de développement** (Tommy + James) : référence pendant la phase de codage (S2 → S12). **Jury Holberton** : preuve de la conception détaillée du MVP.
- **Jury RNCP 5** : portfolio de compétences réutilisables pour la certification Développeur Web et Web Mobile.

1.4 Stack technique

Couche	Technologie	Justification
Front-End	React + Tailwind CSS	SPA performante, design system rapide
Back-End	Django + Django REST Framework (DRF)	Sécurité native, écosystème mature
Base de données	PostgreSQL ou MySQL	Analyse relationnelle ACID, index hautes performances
Asynchrone	Celery + Redis	Tâches longues à développer au-delà du niveau RNCP 5 Développeur Web et Web Mobile
Auth	SimpleJWT	Stateless, évolutif
Hébergement	Vercel + Railway	HTTPS auto, déploiement Git

User Stories & Maquettes

2.1 Personas

Marie : 45 ans, gérante d'un cabinet comptable à Dijon, aucune notion IT. Cible UX < 3 min.

Karim : 32 ans, expert cybersécurité, gère la file de demandes en parallèle.

2.2 User Storie priorisée

User Story	Étape (Semaine)
En tant que visiteur, je veux m'inscrire afin de créer mon compte.	S3-4
En tant qu'utilisateur, je veux me connecter pour accéder à mon compte.	S4
En tant qu'utilisateur, je veux me déconnecter afin de protéger mon compte.	S5
En tant que client PME, je veux soumettre une demande en < 3 min.	S6
En tant que client, je veux suivre le statut (4 statuts).	S7
En tant que client, je veux recevoir un e-mail à chaque changement.	S8
En tant qu'admin, je veux voir toutes les demandes triées.	S9
En tant qu'admin, je veux modifier le statut.	S10
En tant qu'admin, je veux générer un PDF.	S10
En tant que client, je veux télécharger mon PDF.	S10

Should Have

Modules de sensibilisation Réinitialisation du mot de passe Filtres admin
Notifications UI

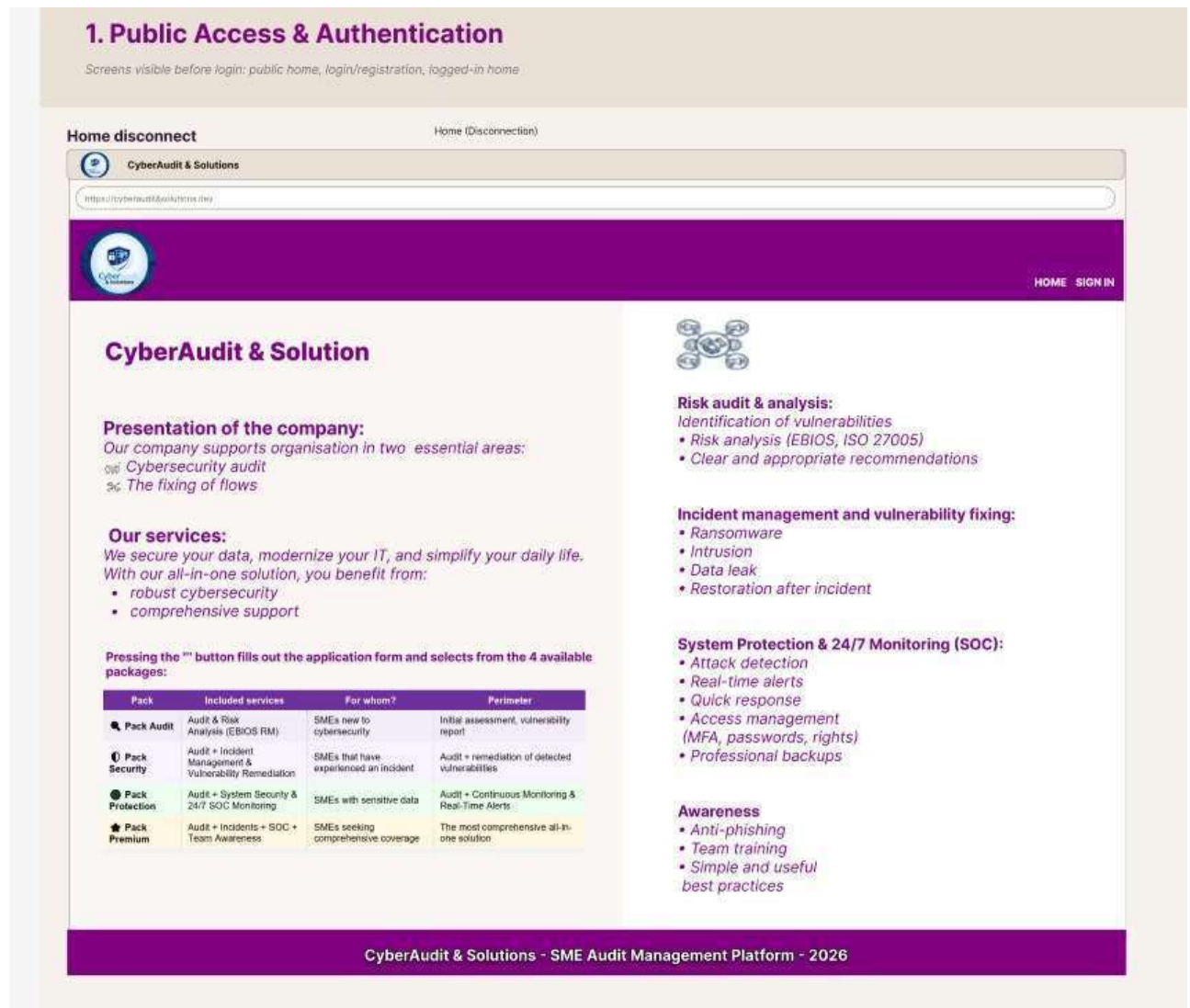
Could Have / Won't Have

Hors périmètre :
Application mobile Scoring IA Marketplace OAuth2 Gamification

2.3 Description des écrans (11 maquettes Figma à produire en S1)

Écran 1 : Page d'accueil publique

La page d'accueil de la plateforme CyberAudit & Solutions présente l'entreprise, ses services et les packs proposés, taillés pour les entrepreneurs de moins de 50 collaborateurs. Il suffit de se rendre dans l'onglet « SIGN IN » pour que la cliente (Marie) puisse créer un compte et se connecter.



Écran 2 : Inscription

Après avoir cliqué sur « Créer un compte », la cliente saisit le nom de son entreprise, son prénom, son nom, son adresse e-mail et son mot de passe (haché).

Règles de remplissage du formulaire :

- Chaque champ contient au maximum 50 caractères.
- Si un champ est manquant, le message d'erreur « Veuillez renseigner les informations du formulaire » s'affiche.
- Format de l'adresse e-mail : nom@domaine requis.
- Mot de passe : une minuscule, une majuscule, un chiffre, un caractère spécial, 10 caractères minimum.

The screenshot displays the 'Create an account' interface for CyberAudit & Solutions. The page has a purple header bar with the company logo on the left and 'HOME SIGN IN' on the right. Below the header, there's a white card with the 'Create an account' title and a subtitle 'Fill in the information below to get started'. The form contains five input fields: 'Company Name' (placeholder: 'Enter company name'), 'First Name' (placeholder: 'Enter first name'), 'Last Name' (placeholder: 'Enter last name'), 'Email' (placeholder: 'Enter email address'), and 'Password' (placeholder: 'Create a password'). A purple 'Create an account' button is positioned below the fields. At the bottom of the card, there's a link 'Already have an account?' and a 'Sign In' button. The footer is a solid purple bar with the text 'CyberAudit & Solutions - SME Audit Management Platform - 2026'.

Écran 3 : Connexion

La cliente peut se connecter avec son compte ; sinon, elle clique sur « Créer un compte ».

Les liens « E-mail oublié » et « Mot de passe oublié » sont prévus (à développer ultérieurement pour la présentation RNCP 5).

The screenshot displays a web application interface for 'CyberAudit & Solutions'. At the top, there is a navigation bar with a 'Sign in' link and a 'Log in' button. Below this is a purple header containing the company logo and the text 'HOME SIGN IN'. The main content area is a light beige color, featuring a central white card with a lock icon and the heading 'Log In'. Below the heading, it says 'Enter your credentials to access your account'. The form includes two input fields: 'Email / User name' with a placeholder 'Enter your email' and 'Password' with a placeholder 'Enter your password'. There are also links for 'Lost Mail ?' and 'Lost Password ?'. A purple 'Connection' button is positioned below the password field. At the bottom of the card, there is a link 'Don't have an account?' and a 'Create an account' button. The footer is a purple bar with the text 'CyberAudit & Solutions - SME Audit Management Platform - 2026'.

Écran 4 : Tableau de bord client

Lorsque la cliente est connectée, elle peut consulter son tableau de bord pour suivre l'avancement de sa demande d'audit (En attente / En cours / Terminé). Elle peut cliquer sur « Voir les détails » et télécharger le rapport via « Télécharger le rapport ».

Client Dashboard

CyberAudit & Solutions

https://cyberaudit3olutions.dev

HOME SIGN OUT

Client Portal

- Dashboard
- Audit Request
- Training

My Dashboard

Welcome Marie - Real-time tracking of your audit requests

Open requests
2

Completed
1

Reports available
1

My audit requests

Filter: All statuses All packs

Case #	Pack	Submission	Status	Action
DOSSIER-2026-0001	Premium	29/04/2026	In Progress	View details
DOSSIER-2026-0002	Audit	22/04/2026	Pending	View details
DOSSIER-2026-0003	Security	10/04/2026	Completed	Download report

Recent notifications

DOSSIER-2026-0001 - Status changed: Pending → In Progress (2h ago)

CyberAudit & Solutions - SME Audit Management Platform - 2026

Écran 5 : Formulaire d'audit

En cliquant sur l'onglet « Demande d'audit », la cliente accède au formulaire de demande d'audit.

Elle doit renseigner :

- Son nom d'utilisateur (50 caractères maximum) Le nom de l'entreprise (moins de 50 caractères)
- Choisir le pack (Audit / Sécurité / Protection / Premium). Lorsque la cliente sélectionne le pack via le bouton radio, les détails du pack s'affichent dans le panneau « Services inclus », notamment « Services inclus », « Pour qui et quels besoins sont adressés ? », « Périmètre d'analyse », « Estimation de l'analyse » et « Tarif ».
- Un message optionnel décrivant la demande d'audit souhaitée.
- Après avoir complété le formulaire, la cliente clique sur « Envoyer la demande d'audit » pour la soumettre.

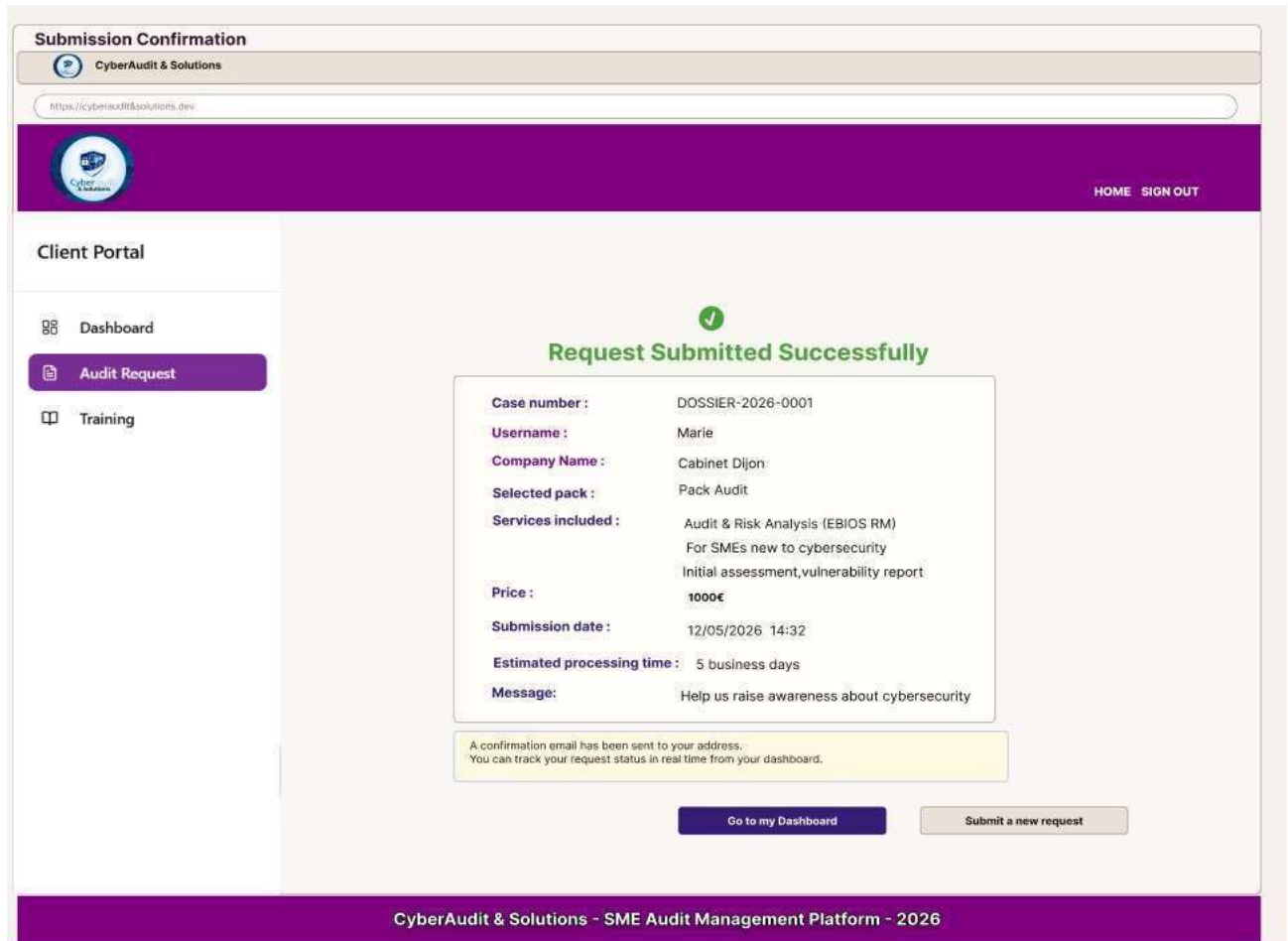
The screenshot shows a web browser window with the URL `https://cyberauditsolutions.dev`. The page title is "CyberAudit & Solutions". The header is purple with a logo on the left and "HOME SIGN OUT" on the right. A left sidebar titled "Client Portal" contains three menu items: "Dashboard", "Audit Request" (highlighted with a purple background), and "Training". The main content area is titled "Audit request form". It contains the following fields and elements:

- Username:** A text input field.
- Company Name:** A text input field.
- Select pack:** Four radio button options: "Pack Audit" (selected), "Pack Security", "Pack Protection", and "Pack Premium".
- Included Services:** A text area displaying details for the selected "Pack Audit":
Print in the panel of included services of the selected package such as:
If you select "Audit & Risk"
Display for example:
Analysis (EBIDS RMI) | SMEs new to cybersecurity | initial assessment/vulnerability report | 5 business day | Price: 1000 €
- Message:** A large text area for an optional message.
- Submit Button:** A purple button labeled "Sent the audit request".

The footer is purple and contains the text "CyberAudit & Solutions - SME Audit Management Platform - 2026".

Écran 6 : Détail de la demande (client)

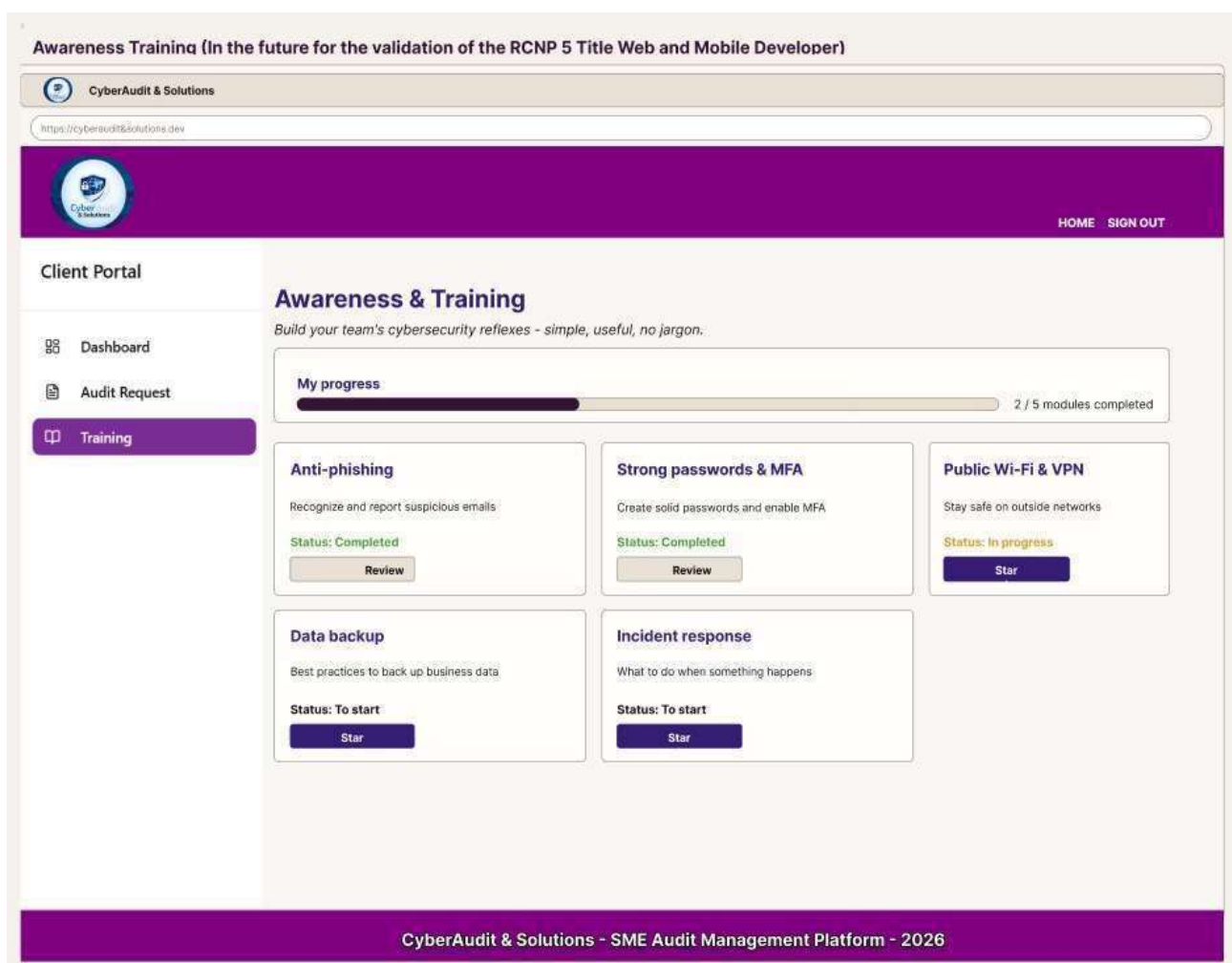
Après l'envoi, un message de confirmation est transmis à l'expert (Karim) et un accusé de réception est envoyé à la cliente. La cliente suit l'avancement de l'analyse.



Écran 7 : Module de formation

À développer ultérieurement. Modules : « Anti-phishing », « Authentification multi-facteurs », « Connexion publique et Wi-Fi & VPN », « Sauvegarde des données », « Réponse aux incidents ». Sous différents formats (vidéo, visioconférence, documentation) avec une barre de progression.

Cette partie de la section sera présentée à l'avenir au-delà du titre RCNP 5 Développeur Web et Web Mobile. L'équipe CyberAudit & Solutions devrait nous aider à préparer les modules de formation dans l'application.



Écran 8 : Tableau de bord admin

L'expert (Karim) accède à son tableau de bord rassemblant toutes les demandes. Il peut trier la liste selon le pack choisi et les statuts en cours, puis consulter et modifier chaque demande.

Admin Dashboard

CyberAudit & Solutions

https://cyberaudit&solutions.dev/

HOME SIGN OUT

Admin Portal

- Dashboard
- Request
- Report

Admin Dashboard - Audit Requests

CyberAudit operator view - Manage all SME requests

Pending: 5

In Progress: 8

Completed: 23

Archived: 12

Filters :

Status: All Pack: All Date: DD/MM/YYYY Client: Search Apply Reset filters

Case #	Client	Pack	Submitted	Status	Action
DOSSIER-2026-0001	Marie SME	Premium	29/04/2026	In Progress	View / Edit
DOSSIER-2026-0002	Tech Corp	Audit	22/04/2026	Pending	View / Edit
DOSSIER-2026-0003	BlueRoof SARL	Security	10/04/2026	Completed	View report
DOSSIER-2026-0004	City Hall Dijon	Protection	03/04/2026	In Progress	View / Edit
DOSSIER-2026-0005	Atelier Pro	Audit	26/03/2026	Archived	View report

5 of 48 requests

1 2 3 4

CyberAudit & Solutions - SME Audit Management Platform - 2026

Écran 9 : Détail de la demande (admin)

En cliquant sur « Voir/modifier », l'expert peut consulter la demande de la cliente, changer le statut via le formulaire « Statut & Actions », générer un rapport PDF via « Générer le rapport PDF », envoyer une notification à la cliente (« Envoyer une notification ») et archiver la demande (« Archiver la demande »)

Admin Request Detail

CyberAudit & Solutions

https://cyberaudit&solutions.dev

HOME SIGN OUT

Admin Portal

Dashboard

Request

Report

Request DOSSIER-2026-0001 [Back to list](#)

Client information

Company : Marie SME (Cabinet Comptable)

Contact : Marie Dupont

Email : marie@cabinet-dijon.fr

Submitted : 29/04/2026 at 14:32

Pack selected : **Pack Audit**

Client message : Help us raise awareness about cybersecurity

Status & Actions

Current status : In Progress

Assigned to : Karim

Internal notes

EBIOS RM analysis ongoing. Awaiting client log files.

[Save changes](#) [Generate PDF report](#) [Send notification](#) [Archive request](#)

History (immutable - GDPR)

Date	Author	Action
29/04/2026 14:32	Marie Dupont	Request submitted (Pack Premium)
29/04/2026 14:32	System	Confirmation email sent (Celery)
29/04/2026 16:10	Karim	Status changed: Pending → In Progress
29/04/2026 16:11	Karim	Internal note added.

CyberAudit & Solutions - SME Audit Management Platform - 2026

Écran 10 : Visualisation du rapport PDF

Le rapport peut être téléchargé. Le rapport identifie :

- Le niveau de vulnérabilités (Faible, Moyen, Élevé)
- Les actifs (serveur de messagerie, routeur Wi-Fi, etc.)
- La description et les recommandations
- Un plan d'action est généré avec le nombre estimé de jours.

PDF Report Viewer

CyberAudit & Solutions

https://cyberaudit&solutions.dev

HOME SIGN OUT

Admin Portal

Dashboard

Request

Report

Vulnerability Report - DOSSIER-2026-0001

[Download PDF](#)

Global security score

B+ Acceptable - 3 medium-severity vulnerabilities found 72 / 100

Executive summary

EBIOS RM analysis performed on the IS perimeter declared by the client. 3 vulnerabilities identified, none critical. Action plan attached, prioritized over 30 days.

Vulnerabilities identified

Severity	Asset	Description	Recommendation
Medium	Email server	No DMARC policy	Activate DMARC + SPF + DKIM
Medium	Wifi router	Default WPA2 password	Rotate password and enable WPA3
Low	Workstation	Outdated antivirus	Update Defender + central console

30-day action plan

Week 1: DMARC/SPF/DKIM activation
 Week 2: Wi-Fi rotation + WPA3 migration
 Week 3: Endpoint update campaign
 Week 4: Final compliance check

CyberAudit & Solutions - SME Audit Management Platform - 2026

Écran 11 : Téléchargement depuis le tableau de bord

La cliente peut télécharger le rapport depuis son tableau de bord via le lien « Télécharger le rapport ».

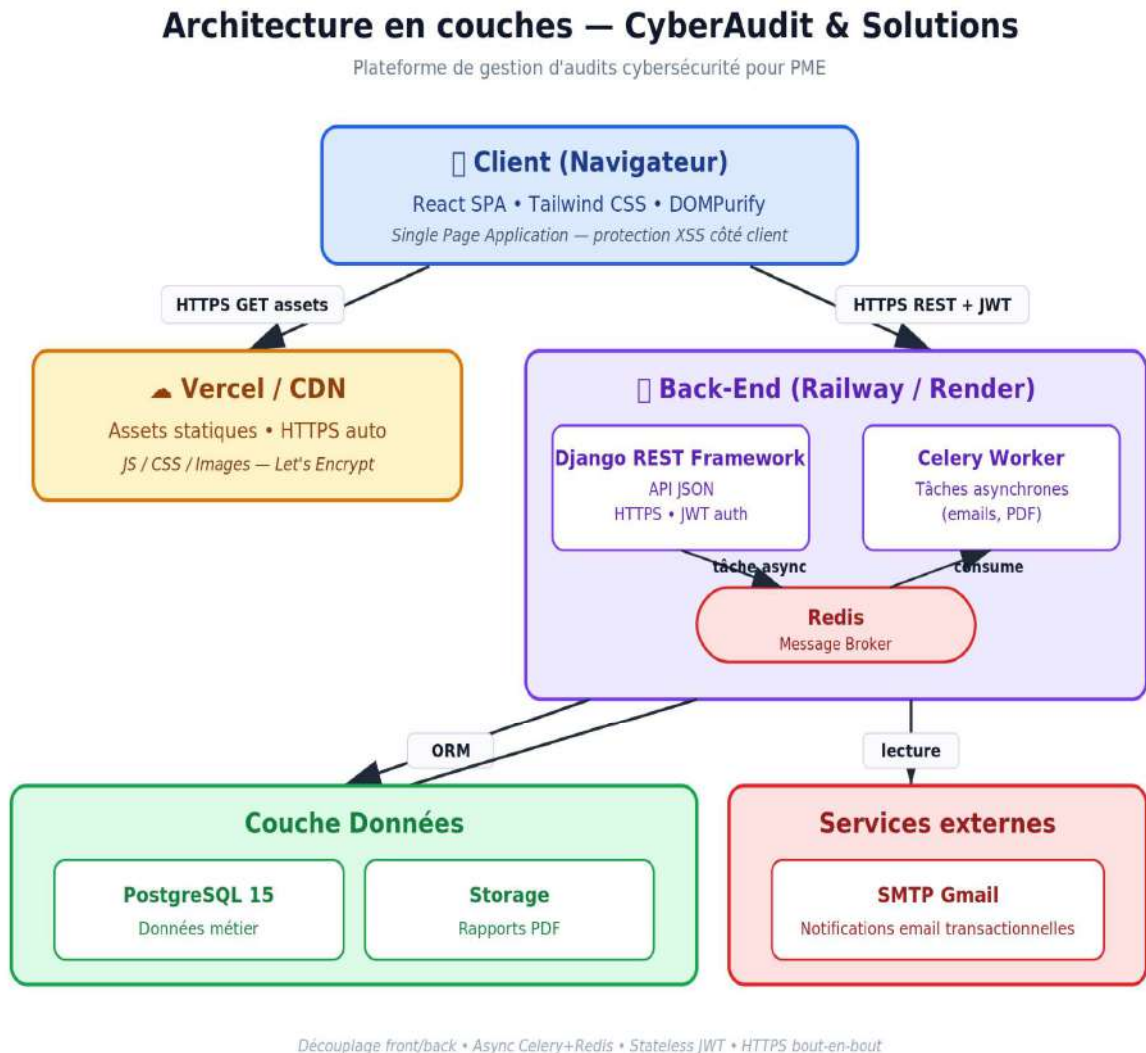
The screenshot displays the 'Client Portal' interface. On the left is a sidebar with 'Dashboard' (selected), 'Audit Request', and 'Training'. The main area is titled 'My Dashboard' with a welcome message for Marie. It features three summary cards: 'Open requests' (2), 'Completed' (1), and 'Reports available' (1). Below these is the 'My audit requests' section, which includes filter dropdowns for 'All statuses' and 'All packs'. A table lists three audit requests. The 'Download report' link for the 'Completed' request is highlighted with a yellow circle.

Case #	Pack	Submission	Status	Action
DOSSIER-2026-0001	Premium	29/04/2026	In Progress	View details
DOSSIER-2026-0002	Audit	22/04/2026	Pending	View details
DOSSIER-2026-0003	Security	10/04/2026	Completed	Download report

Architecture du système

3.1 Vue d'ensemble : architecture en couches

L'architecture suit un modèle en couches :



Architecture en couches : Client (navigateur) → CDN/Back-End → Données + Services externes.

Client (navigateur) :

- React SPA
- Tailwind CSS DOMPurify (anti-XSS)
- HTTPS REST + JWT

Vercel / CDN :

- Assets statiques HTTPS Let's Encrypt

Back-End (Railway / Render) :

- API Django REST (JSON, JWT, HTTPS) Worker Celery (asynchrone)
- Redis (broker)
- ORM SMTP

Couche données :

- PostgreSQL/MySQL + stockage PDF.
- **Services externes** : Gmail SMTP (notifications e-mail).

3.2 Flux de données : Demande d'audit

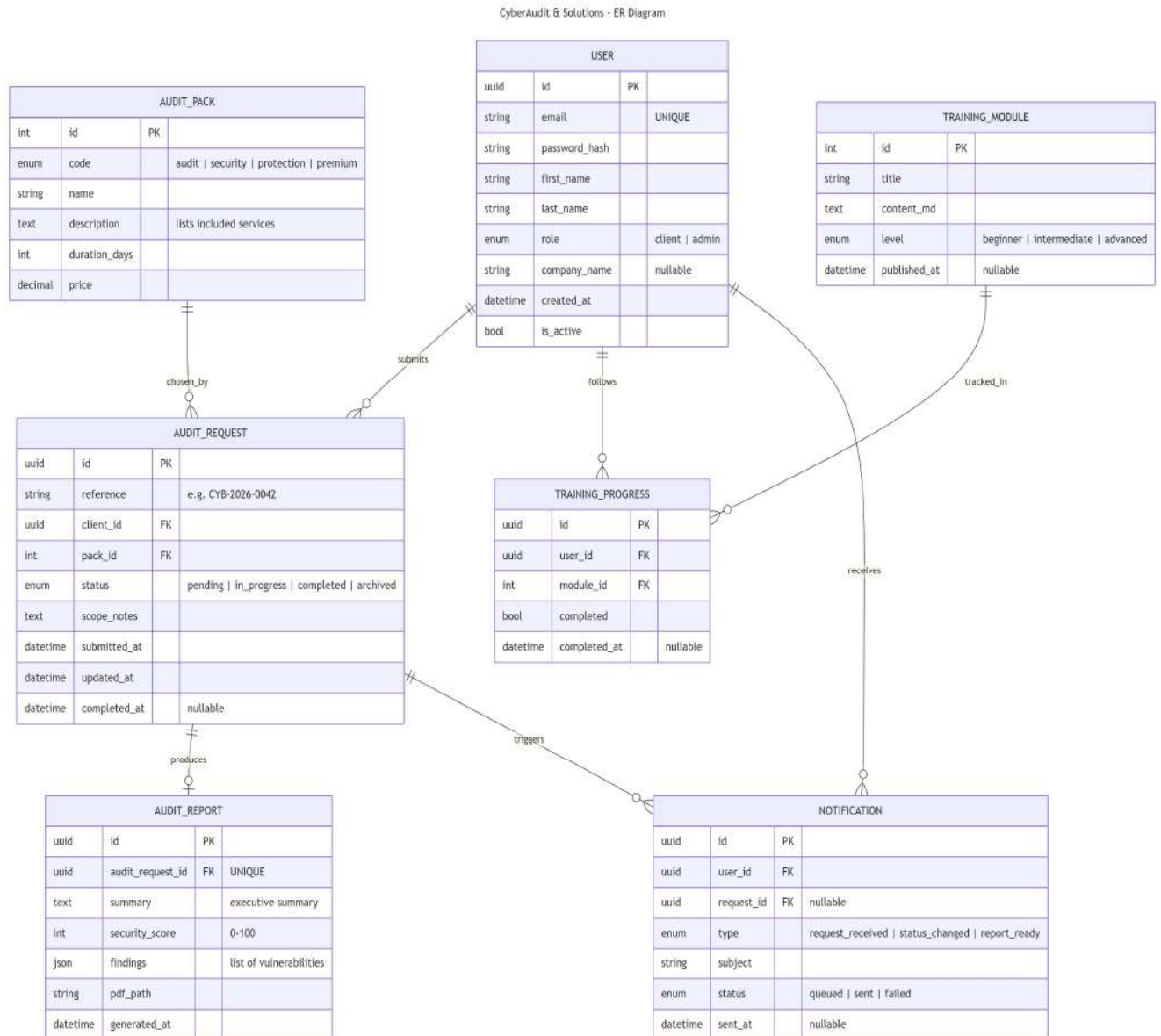
- La cliente remplit le formulaire dans la SPA React.
- Le front envoie POST /api/audits/ (JSON + JWT dans Authorization: Bearer).
- Django valide la requête (serializer DRF) et la persiste en base via l'ORM.
- Django publie une tâche Celery send_acknowledgment_email(audit_id) dans Redis.

3.3 Choix d'architecture justifiés

- **Découplage front/back** → 2 déploiements indépendants, CORS contrôlé, SPA cachable sur CDN.
- **Async Celery** → la génération d'un PDF peut prendre 5-10 s ; on ne bloque pas le thread HTTP.
- **PostgreSQL relationnel** → données fortement structurées avec relations (User → AuditRequest → Report), intégrité ACID critique.
- **JWT stateless** → pas de session serveur, scalable horizontalement.

Composants, classes et base de données

4.1 Schéma de la base de données



Entité	Champs clés	Relation
USER	id (UUID), email (UK), password_hash, first_name, last_name, role (client/admin), company_name, created_at, is_active	1→N AuditRequest (submits / follows / receives)
AUDIT_PACK	id, reference, code (audit/security/protection/premium), name, description, duration_days, price	1→N AuditRequest (chosen_by)
AUDIT_REQUEST	id (UUID), client_id (FK), pack_id (FK), status (pending/in_progress/done/archived), scope_notes, submitted_at, updated_at, completed_at	N→1 User+Pack ; 1→1 Report ; 1→N Notification
AUDIT_REPORT	id (UUID), audit_request_id (FK UK), pdf_path, summary, findings (JSON), generated_at	1→1 AuditRequest
TRAINING_MODULE	id, title, content_md, duration_min, level, published_at	1→N Progress
TRAINING_PROGRESS	id, user_id, module_id, completed, started_at, completed_at	N→1 User + Module
NOTIFICATION	id, user_id, request_id, type, subject, status, sent_at	N→1 User + Request

Recommended indexes:

- USER.email UNIQUE
- AUDIT_REQUEST.status BTREE (frequent admin filtering)
- (AUDIT_REQUEST.client_id, AUDIT_REQUEST.created_at) composite (client dashboard)
- AUDIT_REPORT.audit_request_id UNIQUE (1 report per request)

4.2 Diagramme de classes back-end (modèles Django + services)

Classe User

Représente un utilisateur de la plateforme, qu'il soit client PME ou administrateur. Gère l'authentification JWT et la séparation des rôles.

Attribut	Description
id : UUID	Identifiant unique (PK)
email : str	Adresse e-mail (unique, indexée)
_password_hash : str	Mot de passe haché
first_name, last_name	Prénom et nom
role : enum	Client ou administrateur
company_name : str	Nom de l'entreprise (nullable)
is_active : bool	Compte activé ou désactivé
created_at : datetime	Date de création du compte

Classe AuditPack

Représente l'un des 4 packs (Audit, Sécurité, Protection, Premium). Données de référence chargées via fixture

Attribute	Description
id: int	Identifiant unique
code: enum (str)	audit / sécurité / protection / premium
name: str	Nom commercial
description: str	Liste des services inclus
duration_days: int	Temps de traitement supplémentaire
price: Decimal	Prix du forfait

AuditRequest Class

Cœur de l'entreprise : la demande d'audit soumise par un client PME. Cycle de 4 statuts (en attente → en cours → terminé → archivé).

Attributs	Description
id: UUID	Identifiant unique
reference: str	Numéro de dossier (ex. : CYB-2026-0042)
client_id: UUID FK → User	Client PME ayant soumis
pack_id: UUID FK → AuditPack	Forfait sélectionné
status: str(enum)	en attente / en cours / terminé / archivé
scope_notes: str	Détails du formulaire
submitted_at : datetime	Dates du cycle

Méthodes: generate_reference(), update_status(new_status), estimated_completion()

Classe AuditReport

Rapport de vulnérabilités lié à une seule AuditRequest, comprend un score visuel et un PDF téléchargeable.

Rapport de vulnérabilités lié à une seule AuditRequest, inclut un score visuel et un PDF téléchargeable.

Attributs	Description
id : UUID	Identifiant unique
audit_request_id : FK	Relation 1-1
summary : str	Résumé exécutif vulgarisé
security_score : int	Score 0-100 (jauge)
findings : JSON	Liste des vulnérabilités
pdf_path : str	Chemin du PDF généré
generated_at	Date de génération

Méthodes : build_pdf(), get_download_url()

Classe Notification

Trace toutes les communications e-mail envoyées via Celery.

Attributes	Description
id : UUID	Identifiant unique
user_id : FK → User	Destinataire
request_id : FK	Demande concernée (nullable)
type : enum	request_received / status_changed / report_ready
status : enum	queued / sent / failed

Méthodes : send(), retry()

Classes TrainingModule & TrainingProgress

Modules de sensibilisation cybersécurité (phishing, MFA, RGPD...) avec suivi de progression utilisateur.

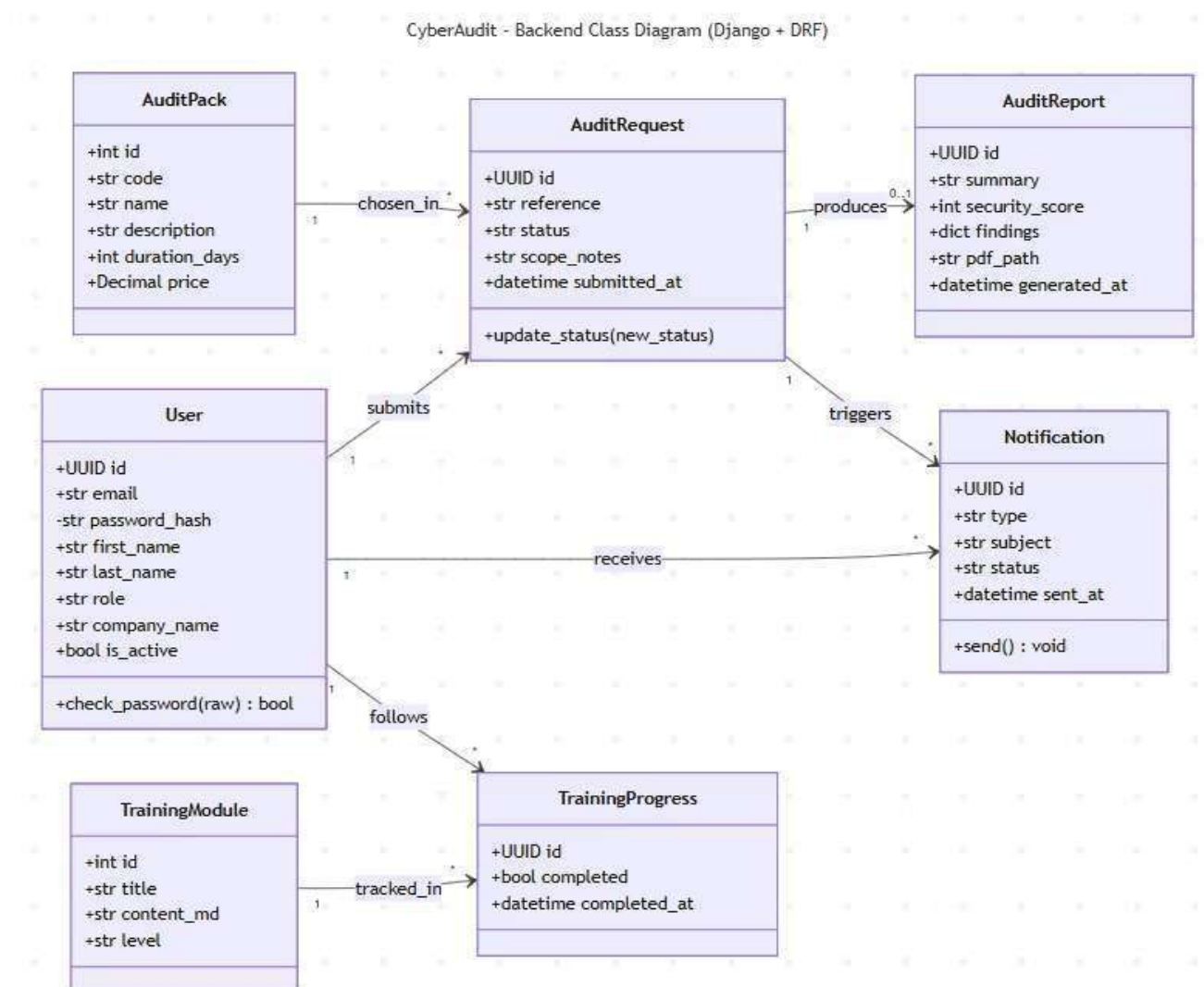


Figure 3 — Diagramme de classes back-end Django

Exemple – accounts/models.py

```
class User(AbstractBaseUser, PermissionsMixin):
    """Utilisateur de la plateforme – client PME ou admin CyberAudit."""
    id = UUIDField(primary_key=True, default=uuid4)
    email = EmailField(unique=True, db_index=True)
    role = CharField(choices=[('client', 'Client'), ('admin', 'Admin')])
    company_name = CharField(max_length=200, blank=True)
    company_size = CharField(choices=[('TPE', 'TPE'), ('PME', 'PME'), ('ETI', 'ETI')])
    sector = CharField(max_length=100, blank=True)
    is_active = BooleanField(default=True)
    created_at = DateTimeField(auto_now_add=True)
    last_login = DateTimeField(null=True)

    USERNAME_FIELD = 'email'
    REQUIRED_FIELDS = []

    def is_admin(self) -> bool: ...
    def is_client(self) -> bool: ...
```

Exemple : audits/models.py

```
class AuditPack(models.Model):
    """Catalogue des 4 packs d'audit proposés."""
    name = CharField(max_length=50)
    description = TextField()
    duration_days = PositiveIntegerField()
    price = DecimalField(max_digits=8, decimal_places=2)

class AuditRequest(models.Model):
    """Demande d'audit soumise par un client."""
    STATUS_CHOICES = [
        ('pending', 'En attente'),
        ('in_progress', 'En cours'),
        ('done', 'Terminé'),
        ('archived', 'Archivé'),
    ]
    id = UUIDField(primary_key=True, default=uuid4)
    client = ForeignKey(User, on_delete=PROTECT, related_name='audit_requests')
    pack = ForeignKey(AuditPack, on_delete=PROTECT)
    status = CharField(choices=STATUS_CHOICES, default='pending', db_index=True)
    scope_notes = TextField(blank=True)
    created_at = DateTimeField(auto_now_add=True)
    updated_at = DateTimeField(auto_now=True)
    completed_at = DateTimeField(null=True, blank=True)

    def transition_to(self, new_status: str) -> None:
        """Change le statut, met à jour completed_at et déclenche la notification.
        """
        ...

    def can_generate_report(self) -> bool:
        return self.status == 'done'
```

Services métier (logique séparée des modèles)

```
# audits/services.py
class AuditRequestService:
    @staticmethod
    def create_request(client: User, pack_id: int, scope: str) -> AuditRequest: ..
    .

    @staticmethod
    def change_status(request: AuditRequest, new_status: str, by: User) -> None:
        """Change le statut + déclenche email Celery."""
        ...

# reports/services.py
class PDFReportGenerator:
    @staticmethod
    def generate(audit_request: AuditRequest) -> AuditReport:
        """Génère un PDF avec WeasyPrint, stocke le chemin, persiste."""
        ...

# notifications/tasks.py (Celery)
@shared_task
def send_status_change_email(audit_id: str, new_status: str) -> None: ...
```

4.3 Composants front-end (React + Tailwind CSS)

Le front-end est une SPA React communiquant avec l'API REST via JWT. L'arborescence se structure en pages (vues complètes), composants UI réutilisables, store global et services d'appel API.

Authentification & routage

AuthStore — State global : user, accessToken, refreshToken. Méthodes : login, logout, refresh, hasRole.

ProtectedRoute — Composant de garde, props : allowedRoles. Redirige vers /login si non authentifié.

ApiClient — Wrapper axios qui ajoute automatiquement le JWT à chaque requête et gère le refresh token en cas de 401.

Pages publiques

LoginPage — Formulaire de connexion. Soumet à AuthStore.login() puis redirige selon le rôle.

RegisterPage — Formulaire d'inscription client SME. Appelle AuthStore.register().

Espace client SME

ClientDashboard — Tableau de bord listant les demandes du client connecté.

AuditRequestForm — Formulaire en 3 étapes (pack, périmètre, confirmation). Affiche 4 PackCard.

RequestDetail — Vue détaillée avec timeline, rapport PDF, score. Composants : StatusBadge, SecurityScoreGauge, FindingsList.

Espace administrateur CyberAudit

AdminDashboard — Vue paginée de toutes les demandes avec filtres (statut, pack, date, client) et tri.

Module de formation

TrainingPage — Liste des modules avec indicateur de progression.

TrainingModuleViewer — Affiche le contenu Markdown. Sécurité : nettoyage via DOMPurify (anti-XSS, SMART #3).

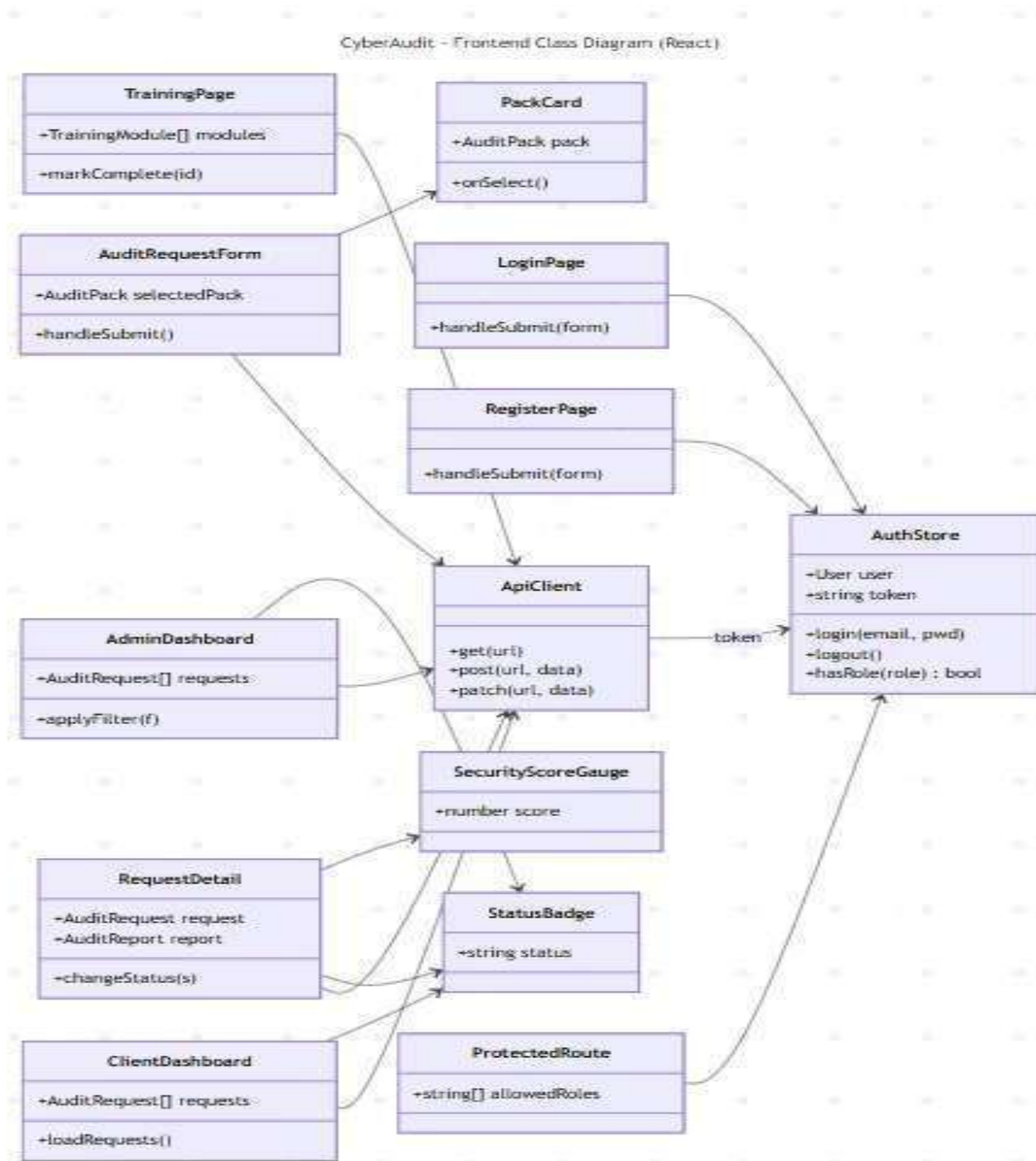
Composants UI réutilisables

Navbar, **PackCard**, **StatusBadge** (gris/bleu/vert/violet), **SecurityScoreGauge** (jauge circulaire, rouge < 40, orange 40-70, vert > 70), **RequestTable**, **Pagination**, **ToastContainer**.

Flux d'interaction principal : Soumission demande

1. L'utilisateur se connecte via **LoginPage** → **AuthStore** stocke le JWT.
2. **ProtectedRoute** autorise l'accès au **ClientDashboard**.
3. Clic sur « Nouvelle demande » → **AuditRequestForm** charge les packs.
4. Sélection d'une **PackCard** → **AuditService.create()** appelle l'API Django.
5. Le back-end crée l'**AuditRequest** et déclenche la tâche Celery d'email.
6. Le client est redirigé vers **RequestDetail** (**StatusBadge** "pending").
7. À chaque changement de statut côté admin, une Notification est envoyée.

CyberAudit – Frontend Class Diagram (React) Explained



Ce diagramme montre l'architecture frontend React d'une application d'audit de cybersécurité appelée CyberAudit. Il illustre les principaux composants UI (classes) et la façon dont ils dépendent de services partagés. Voici une explication détaillée.

Services centraux

Deux classes se trouvent au cœur de l'application et sont référencées par presque tout le reste :

ApiClient est la couche HTTP. Elle expose les méthodes **get(url)**, **post(url, data)** et **patch(url, data)** que le reste de l'application utilise pour communiquer avec le back-end. Elle lit le jeton d'authentification depuis **AuthStore** (visualisé par la flèche étiquetée « token ») afin que les requêtes soient authentifiées automatiquement.

AuthStore conserve l'état de session : l'utilisateur courant, le jeton de type JWT, et les méthodes **login(email, pwd)**, **logout()** et **hasRole(role): bool**. Les composants l'utilisent pour savoir qui est connecté et ce qu'il est autorisé à faire.

Authentification et routage

LoginPage et **RegisterPage** exposent chacune une méthode **handleSubmit(form)**. Elles dépendent toutes deux d'**ApiClient** (pour envoyer les identifiants au back-end) et d'**AuthStore** (pour persister l'utilisateur/ jeton après une réponse réussie).

ProtectedRoute est un garde-route avec une propriété **allowedRoles: string[]**. Il interroge **AuthStore.hasRole(...)** pour décider d'afficher la route enfant ou de rediriger — c'est ainsi que l'application applique le contrôle d'accès basé sur les rôles.

Écrans destinés au client

ClientDashboard contient **AuditRequest[]** et une méthode **loadRequests()**. Il appelle **ApiClient** pour récupérer les demandes d'audit du client connecté et utilise **StatusBadge** pour afficher leur état.

AuditRequestForm permet à un client de soumettre un nouvel audit. Il contient un **selectedPack: AuditPack** et une méthode **handleSubmit()**, et envoie le formulaire au back-end via **ApiClient**. Il affiche également des composants **PackCard** chaque **PackCard** affiche un **AuditPack** et expose un callback **onSelect()** pour que l'utilisateur puisse choisir un pack.

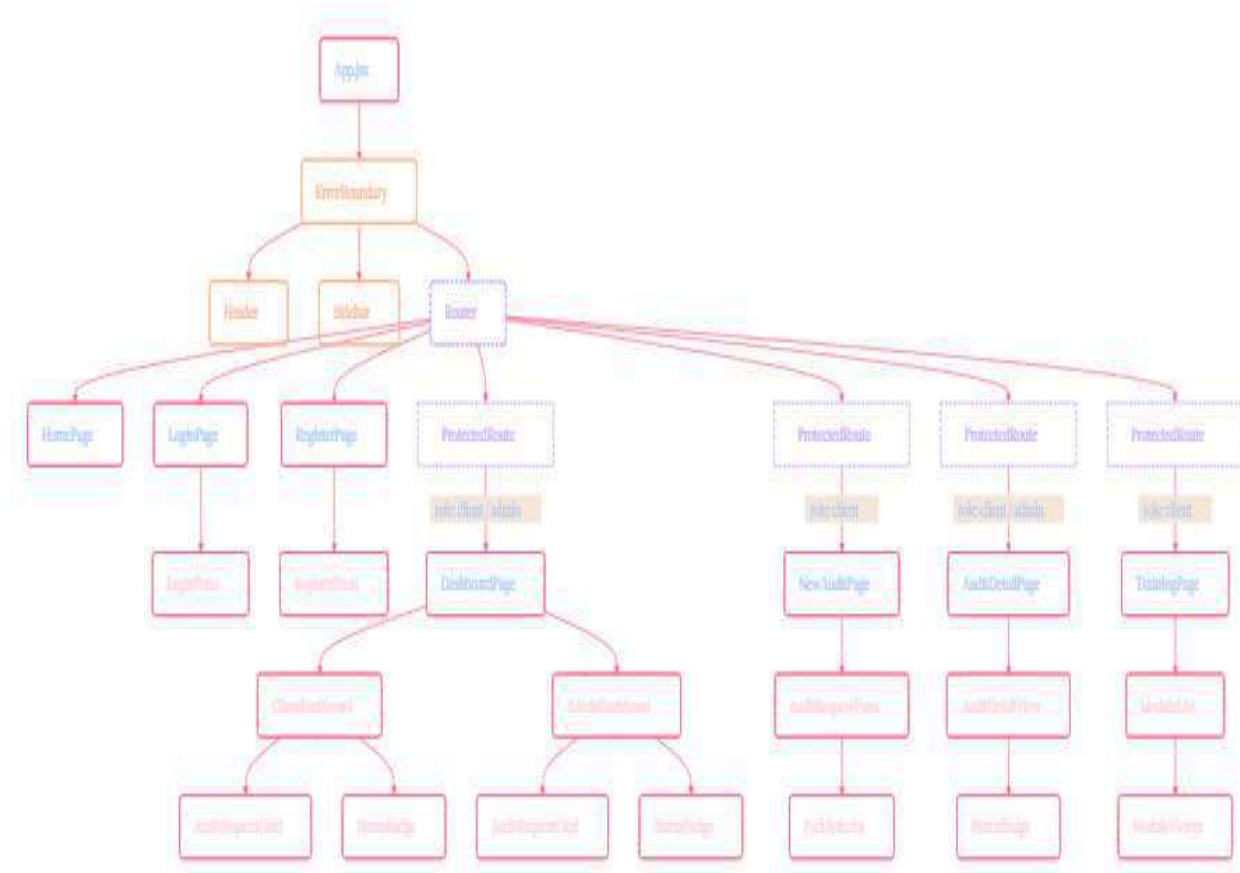
RequestDetail est la vue par demande. Elle contient un **AuditRequest** et un **AuditReport**, expose **changeStatus(s)** et compose deux widgets réutilisables : **SecurityScoreGauge** (qui affiche un score numérique) et **StatusBadge** (qui affiche un statut sous forme de chaîne). Elle communique avec le back-end via **ApiClient**.

TrainingPage affiche **TrainingModule[]** avec une méthode **markComplete(id)**, la section formation/ sensibilisation de l'application, également soutenue par **ApiClient**.

Côté administrateur

AdminDashboard reflète le tableau de bord client mais pour les administrateurs. Il contient **AuditRequest[] requests** et une méthode **applyFilter(f)**, récupère les données via **ApiClient** et réutilise **StatusBadge** pour l'affichage

Component tree



This diagram shows the component hierarchy of a web application, likely an audit platform with a training module. Read it top to bottom: each arrow means a component contains (or renders) another one.

Ce diagramme montre la hiérarchie des composants d'une application web, probablement une plateforme d'audit avec un module de formation. Lisez-le de haut en bas : chaque flèche signifie qu'un composant contient (ou affiche) un autre. En haut, **App.jsx** est le point d'entrée de l'application. Il englobe tout à l'intérieur d'un **ErrorBoundary**, un composant qui attrape les erreurs JavaScript de ses enfants afin que l'application entière ne plante pas, une bonne pratique standard de React. Sous l'**ErrorBoundary**, vous avez trois composants structurels : le Header (barre supérieure), le Sidebar (menu latéral) et le Router, qui est le cœur de la navigation. Le Header et le Sidebar sont toujours visibles, tandis que le Router décide quoi afficher dans la zone de contenu principal en fonction de l'URL.

Le routeur envoie ensuite vers différentes pages :

D'abord, les pages publiques, accessibles à tous : **HomePage** (page d'accueil), **LoginPage** qui contient un **LoginForm**, et **RegisterPage** qui contient un **RegisterForm**.

Ensuite, les pages protégées, enveloppées dans des composants **ProtectedRoute**. C'est un modèle classique :

- **ProtectedRoute** vérifie que l'utilisateur est authentifié et possède le bon rôle avant d'afficher la page. Il y en a quatre, chacun avec des restrictions de rôle différentes :
 - **DashboardPage** (rôles client / admin) — affiche soit **ClientDashboard** soit **AdminDashboard** en fonction du rôle. Les deux réutilisent les mêmes éléments de base : **AuditRequestCard** et **StatusBadge**. Un bon exemple de composants réutilisables.

- **NewAuditPage** (rôle client uniquement) — contient un `AuditRequestForm` avec un `FieldSelector` pour choisir les champs d'audit.
- **AuditDetailPage** (rôles client / admin) — affiche un `AuditDetailView` qui réutilise également le `StatusBadge`.
- **TrainingPage** (rôle client) — affiche un `ModuleList` qui rend des instances de `ModuleViewer` pour chaque module de formation.

Quelques points intéressants à noter sur l'architecture :

La réutilisabilité est bien gérée : **StatusBadge** est utilisé à trois endroits différents (**ClientDashboard**, **AdminDashboard**, **AuditDetailView**), et **AuditRequestCard** est partagé entre les deux tableaux de bord. C'est un signe d'une bonne décomposition.

La séparation des préoccupations est claire : les pages (*Page) gèrent le routage et les autorisations, tandis que les vues (*View, *Dashboard, *Form) gèrent le rendu et la logique métier.

Le contrôle d'accès est centralisé dans **ProtectedRoute** plutôt que dispersé sur chaque page, ce qui rend la sécurité beaucoup plus facile à auditer.

src/	
├ components/	
│ └ auth/	LoginForm, RegisterForm, ProtectedRoute
│ └ dashboard/	ClientDashboard, AdminDashboard, AuditRequestCard, StatusBadge
│ └ audit/	AuditRequestForm, PackSelector, AuditDetailView
│ └ training/	ModuleList, ModuleViewer
│ └ shared/	Header, Sidebar, Button, ErrorBoundary
├ pages/	HomePage, LoginPage, RegisterPage, DashboardPage, NewAuditPage, AuditDetailPage, TrainingPage
├ hooks/	useAuth, useAudits, useApi
├ services/	api.js (axios + JWT), auth/audits/reports services
├ utils/	sanitize.js (DOMPurify), validators.js
└ App.jsx	routing global

Conventions: Functional components + hooks (no classes), global state via Context API, `sanitize()` on all inputs.

4.4 Génération du rapport PDF de vulnérabilités

Le rapport PDF est le livrable principal du client SME : il consolide l'analyse de vulnérabilités, le score de sécurité et le plan d'action recommandé. Sa génération est entièrement asynchrone pour ne pas bloquer l'interface admin.

Stack technique

- **WeasyPrint** : Convertit un template HTML+CSS en PDF de qualité impression. Réutilise la charte graphique du web.
- **Celery** : Exécute la tâche `generate_pdf_task` en arrière-plan.
- **Redis** : File d'attente Celery (scaling parallèle).
- **Stockage** : `/media/reports/` au MVP, S3 en V2 ; accès via URL signée 15 min.

Structure du PDF généré

Section	Contenu
Page 1 — Couverture	Logo CyberAudit, nom client, référence dossier (CYB-2026-0042), date, confidentialité
Page 2 — Synthèse exécutive	Score global A à F (jauge 0-100) + résumé en 3 lignes vulgarisées
Page 3 — Périmètre audité	Liste des actifs analysés (serveur mail, routeur Wi-Fi, postes...) + méthodologie EBIOS RM
Pages 4-N — Vulnérabilités	Tableau findings : sévérité (Critical/High/Medium/Low), actif, description, recommandation
Page finale — Plan d'action 30 jours	Étapes hebdomadaires (S1, S2, S3, S4) avec actions concrètes et priorités
Annexe	Glossaire, références (ISO 27005, ANSSI), contact CyberAudit

Flux complet : de la demande au téléchargement

- L'admin clique sur « Generate PDF report » dans AdminRequestDetail.
- Le front appelle POST /api/requests/{id}/generate-report avec les findings.
- Le PdfReportService Django crée l'enregistrement AuditReport (résumé, score, findings JSON).
- Il enfile la tâche generate_pdf_task.delay(report_id) et retourne 202 Accepted.
- Le worker Celery rend report.html avec les données, puis WeasyPrint produit le PDF.
- Fichier sauvegardé dans /media/reports/CYB-2026-0042.pdf, pdf_path mis à jour.
- Notification de type report_ready créée, e-mail envoyé via send_email_task.
- Côté client, ClientDashboard affiche « Download report » → GET /api/reports/{id}/download en streaming.

Sécurité d'accès au PDF

- **JWT obligatoire** sur /api/reports/{id}/download. Aucun accès anonyme.
- **Permission IsAdminOrOwner** : 403 Forbidden sinon.
- **URL signée** avec timestamp, expire après 15 minutes.
- **Pas de cache navigateur** : Cache-Control: no-store, private.
- **Audit log (V2)** : qui, quand, depuis quelle IP — conforme RGPD.

Gestion des échecs de génération

- **Retry automatique** : autoretry_for=(Exception,), max_retries=3, backoff exponentiel (10s, 30s, 90s).
- **Statut intermédiaire** : pdf_path vide tant que non généré → front affiche « Génération en cours... ».
- **Notification d'échec** : après 3 échecs, type generation_failed envoyée à l'admin.
- **Régénération manuelle** via le bouton « Generate PDF report ».

4.5 Monitoring & observabilité

L'application doit rester traçable en production : on doit savoir *qui a fait quoi, quand, et pourquoi ça a planté*. La stratégie repose sur 3 piliers : logs structurés, monitoring d'erreurs, alerting.

4.5.1 Logs structurés (JSON)

Django configure structlog pour produire des logs JSON consommables par Railway et un futur ELK/Loki. Chaque log porte au minimum : timestamp, level, request_id (corrélation), user_id, endpoint, latency_ms.

```
{
  "timestamp": "2026-05-18T09:24:11Z",
  "level": "INFO",
  "event": "audit_request.created",
  "request_id": "req-7f2a...",
  "user_id": "8f2a...c9",
  "audit_id": "CYB-2026-0042",
  "pack": "security",
  "latency_ms": 142
}
```

4.5.2 Monitoring d'erreurs — Sentry

- **Back-end** : sentry-sdk[django, celery] capture toutes les exceptions Django et Celery avec stack trace, context utilisateur, payload de la requête (PII anonymisée).
- **Front-end** : @sentry/react capture les erreurs JS et les rejets de promesse non gérés, plus les sessions replay sur erreur.
- **Release tracking** : chaque tag git pousse une release Sentry → suivi de régression par version.
- **Sample rate** : 100 % des erreurs en MVP ; 10 % des transactions performance pour rester dans le quota gratuit.

4.5.3 Métriques business & techniques

Métrique	Source	Seuil d'alerte
Temps de réponse API (p95)	Sentry Performance / Railway metrics	> 800 ms pendant 5 min
Taux d'erreur 5xx	Sentry	> 1 % sur 10 min
Échecs Celery consécutifs	Logs structurés	≥ 3 sur la même tâche
Connexions BDD actives	Railway Postgres metrics	> 80 % du pool
Espace disque /media	Railway	> 85 %
Demandes d'audit / jour	Django + dashboard admin	Métrique de croissance, pas d'alerte

4.5.4 Alerting

- **Sentry** envoie un e-mail au référent technique sur toute nouvelle erreur non vue, et un Slack DM (V2) si taux d'erreur > seuil.

- **Healthcheck** : endpoint GET /api/health/ vérifie BDD, Redis, SMTP. Surveillé par UptimeRobot (gratuit) toutes les 5 min.
- **Notification d'échec Celery** : après 3 retries, un e-mail admin est envoyé (déjà documenté en §4.4).

4.5.5 Audit log applicatif (V2)

Une table AUDIT_LOG trace les actions sensibles : connexions, téléchargement de rapport, changement de statut, suppression de compte. Champs : id, user_id, action, resource, ip, user_agent, created_at. Conservation 1 an, conforme RGPD (cf. §7.5).

4.6 Migrations BDD & seeding

Le passage d'une version de schéma à l'autre, et le pré-remplissage des données de référence (4 packs d'audit, modules de formation initiaux, compte admin) sont automatisés. Aucune intervention manuelle en production.

4.6.1 Stratégie de migrations Django

- **Convention** : 1 migration = 1 changement atomique. Nom explicite : 0007_add_security_score_to_auditreport.py.
- **Schema migrations** : générées via python manage.py makemigrations, jamais éditées à la main.
- **Data migrations** : utilisent migrations.RunPython(forward, reverse) avec une fonction reverse systématique pour permettre le rollback.
- **Exécution en CI** : python manage.py migrate --check dans le pipeline pour détecter les migrations non générées.
- **Production** : Railway lance migrate automatiquement au déploiement via le release command du Procfile.

4.6.2 Exemple de data migration

```
# audits/migrations/0002_seed_packs.py
from django.db import migrations

def seed_packs(apps, schema_editor):
    AuditPack = apps.get_model('audits', 'AuditPack')
    AuditPack.objects.bulk_create([
        AuditPack(code='audit', name='Audit', duration_days=5, price=49
0),
        AuditPack(code='security', name='Security', duration_days=10, price=99
0),
        AuditPack(code='protection', name='Protection', duration_days=15, price=14
90),
        AuditPack(code='premium', name='Premium', duration_days=20, price=24
90),
    ])

def remove_packs(apps, schema_editor):
    apps.get_model('audits', 'AuditPack').objects.all().delete()

class Migration(migrations.Migration):
    dependencies = [('audits', '0001_initial')]
    operations = [migrations.RunPython(seed_packs, remove_packs)]
```

4.6.3 Fixtures (données de démo)

Des fixtures JSON dans backend/fixtures/ permettent de charger des données de démo (5 clients fictifs, 10 demandes d'audit, 3 rapports) pour les démos jury et les tests E2E :

```
python manage.py loaddata fixtures/demo_clients.json
python manage.py loaddata fixtures/demo_audits.json
python manage.py loaddata fixtures/demo_reports.json
```

Un script wrapper scripts/seed_demo.sh enchaîne migrate + loaddata pour un environnement de démo en 1 commande.

4.6.4 Création de l'admin initial

Aucun mot de passe en clair dans le code. La commande Django personnalisée create_initial_admin lit ADMIN_EMAIL et ADMIN_PASSWORD depuis les variables d'environnement et crée l'utilisateur si la table est vide :

```
python manage.py create_initial_admin
# Idempotent : ne fait rien si un admin existe déjà
```

4.6.5 Stratégie de rollback

Scénario	Procédure
Migration cassée juste après déploiement	python manage.py migrate audits 0006 revient à la migration précédente. La fonction reverse de RunPython est exécutée.
Données corrompues détectées	Restauration du backup Railway le plus récent (1 clic), puis investigation hors prod sur snapshot.
Migration nécessitant downtime	Maintenance window planifiée, page de maintenance activée via Vercel, migration exécutée, vérification, réouverture.
Changement de schéma non rétro-compatible	Stratégie « expand & contract » : 1) ajouter la nouvelle colonne, 2) double-écriture côté app, 3) backfill, 4) basculer la lecture, 5) supprimer l'ancienne colonne.

4.6.6 Snapshot avant chaque release

Avant chaque merge sur main (release prod), un snapshot manuel de la BDD Railway est déclenché via *railway db snapshot create*. Conservation 14 jours. Permet un rollback < 5 min en cas de problème majeur.

Diagrammes de séquence

5.1 Inscription + Connexion JWT

L'utilisateur saisit son email et son mot de passe. Le front appelle POST /api/auth/register/, puis POST /api/auth/login/. Django valide les credentials, génère access + refresh tokens, et retourne le tout au front. Le front stocke les tokens dans AuthStore et redirige vers le dashboard correspondant au rôle.

Séquence 1 — Inscription + Connexion JWT

Use case 1 — création de compte puis authentification (access + refresh tokens)

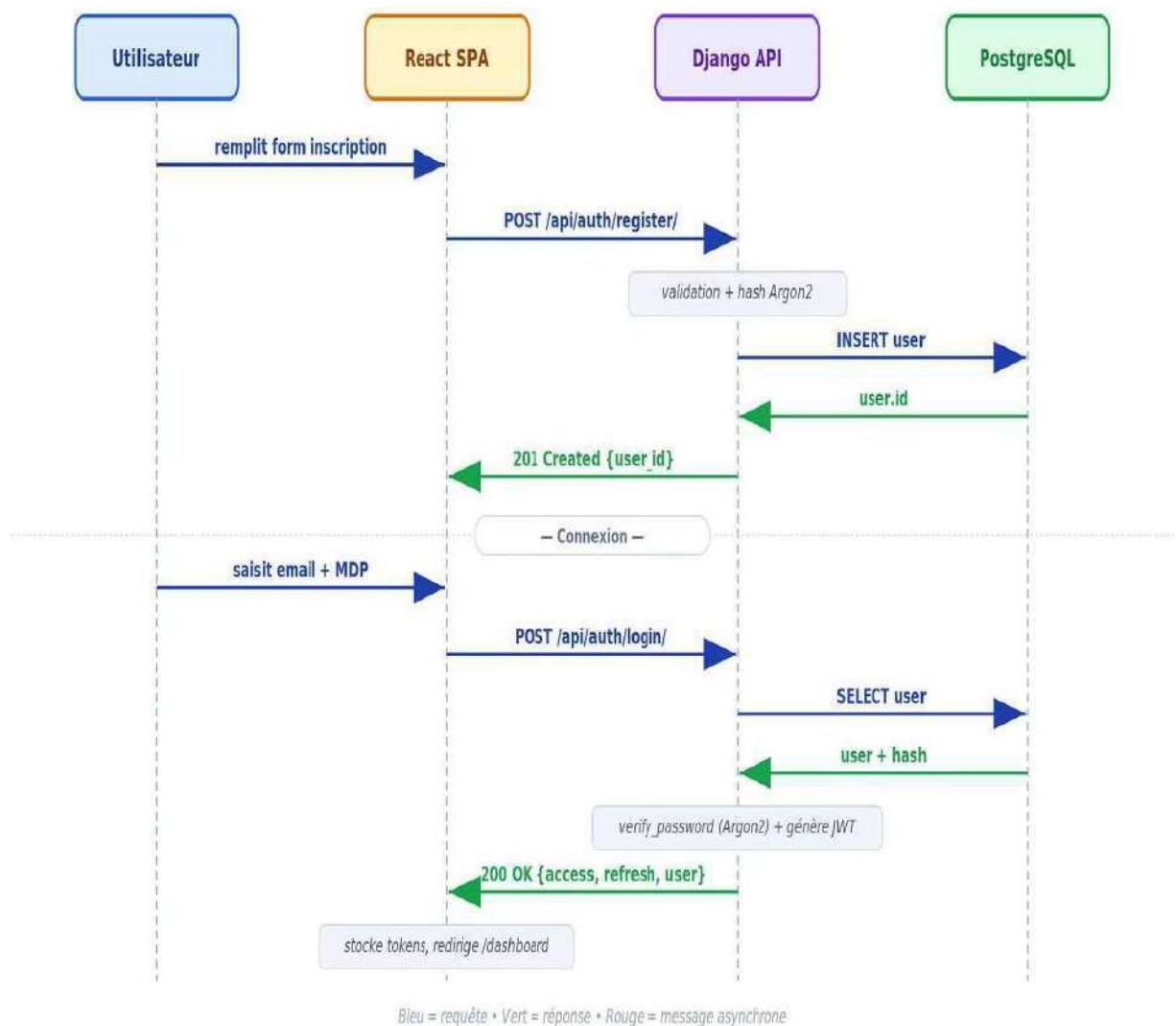


Figure 5 — Séquence Inscription + Connexion JWT

Messages échangés (en détail)

1. **Marie → RegisterPage** : remplit le formulaire (email, password, company_name).
2. **RegisterPage → ApiClient** : POST /api/auth/register/ avec body JSON {email, password, company_name, company_size, sector}.
3. **ApiClient → Django (RegisterView)** : DRF serializer valide les champs (email unique, password robustesse).
4. **Django → PostgreSQL** : INSERT INTO accounts_user avec password haché bcrypt.
5. **Django → Celery** : send_welcome_email.delay(user_id).
6. **Django → RegisterPage** : 201 Created avec {user, access, refresh}.
7. **RegisterPage → AuthStore** : stocke accessToken et refreshToken.
8. **RegisterPage → React Router** : redirection vers /dashboard selon user.role.
9. **Celery worker → SMTP Gmail** : envoie l'e-mail de bienvenue (asynchrone, hors thread HTTP).

Mermaid (rendu sur GitHub)

```
sequenceDiagram
    actor Marie
    participant Front as RegisterPage
    participant API as Django /auth
    participant DB as PostgreSQL
    participant Celery
    participant SMTP as Gmail SMTP

    Marie->>Front: Remplit formulaire
    Front->>API: POST /auth/register/ {email, password, ...}
    API->>API: Valider (DRF serializer)
    API->>DB: INSERT user (password bcrypt)
    API->>Celery: send_welcome_email.delay(user_id)
    API-->>Front: 201 {user, access, refresh}
    Front->>Front: AuthStore.set(tokens)
    Front-->>Marie: Redirect /dashboard
    Celery->>SMTP: Bienvenue email
    SMTP-->>Marie: E-mail reçu
```

5.2 Soumission demande d'audit (avec async)

Le client soumet le formulaire d'audit. Le front appelle POST /api/audits/ avec le JWT en header. Django valide via le serializer DRF, persiste en BDD, enfile une tâche Celery d'envoi d'accusé de réception, puis répond 201 Created. Le worker Celery dépile la tâche depuis Redis et envoie l'e-mail via SMTP.

Séquence 2 — Soumission d'une demande d'audit

Use case 2 — flux asynchrone : réponse immédiate au client + email envoyé en arrière-plan

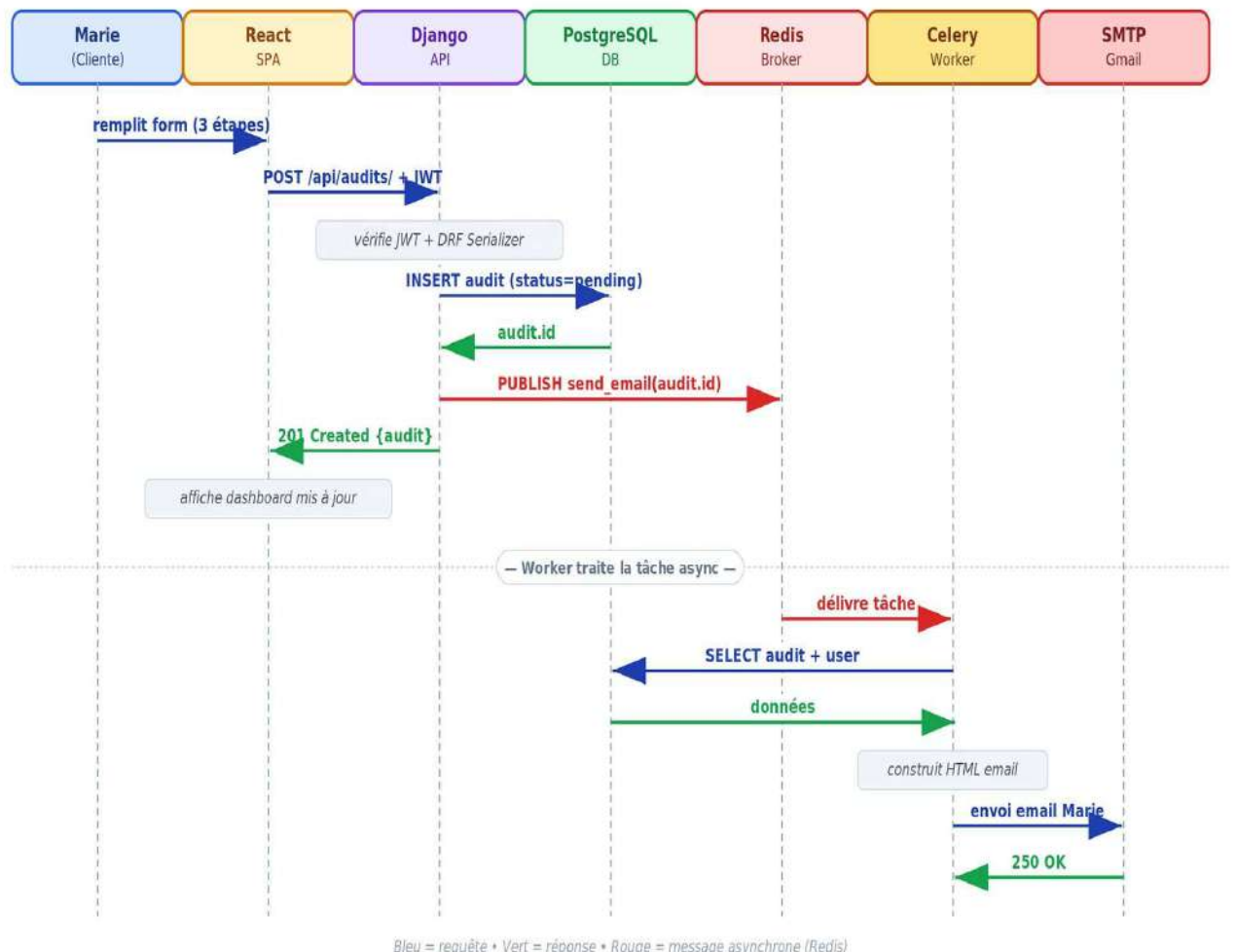


Figure 6 — Séquence Soumission demande (asynchrone Celery)

Messages échangés (en détail)

1. **Marie → AuditRequestForm** : sélectionne un pack et saisit scope_notes.
2. **AuditRequestForm → ApiClient** : POST /api/audits/, header Authorization: Bearer <JWT>, body {pack_id, scope_notes}.
3. **ApiClient → Django (AuditCreateView)** : middleware vérifie le JWT, DRF valide les champs.
4. **Django → PostgreSQL** : INSERT INTO audits_auditrequest avec status='pending', reference='CYB-2026-XXXX'.
5. **Django → Redis (Celery broker)** : enqueue send_acknowledgment_email.delay(audit_id).
6. **Django → AuditRequestForm** : 201 Created avec le détail de la demande.
7. **AuditRequestForm → ClientDashboard** : redirection avec toast « Demande envoyée ».
8. **Celery worker ← Redis** : dépile la tâche.
9. **Celery worker → SMTP Gmail** : envoie l'accusé de réception à Marie + notification à Karim.
10. **Celery worker → PostgreSQL** : UPDATE notifications SET status='sent'.

Mermaid (rendu sur GitHub)

```
sequenceDiagram
    actor Marie
    participant Front as AuditRequestForm
    participant API as Django /audits
    participant DB as PostgreSQL
    participant Redis
    participant Worker as Celery Worker
    participant SMTP

    Marie->>Front: Choisit pack + scope_notes
    Front->>API: POST /audits/ + JWT
    API->>API: Vérifier JWT + valider
    API->>DB: INSERT audit_request (status=pending)
    API->>Redis: enqueue send_ack_email(audit_id)
    API-->>Front: 201 Created {audit}
    Front-->>Marie: Redirect dashboard + toast
    Worker->>Redis: dequeue task
    Worker->>SMTP: Email accusé de réception
    Worker->>DB: UPDATE notification status=sent
    SMTP-->>Marie: Email reçu
```

5.3 Génération du rapport PDF (admin)

L'admin clique sur « Generate PDF report ». Le front appelle POST /api/audits/{id}/generate-report/. Django crée l'AuditReport et enfile la tâche Celery, retournant 202 Accepted. Le worker Celery rend le template HTML, WeasyPrint produit le PDF, le fichier est stocké, et une notification e-mail est envoyée au client.

Séquence 3 – Génération du rapport PDF (admin)

Use case 3 – l'admin déclenche, le worker génère, le client est notifié et récupère

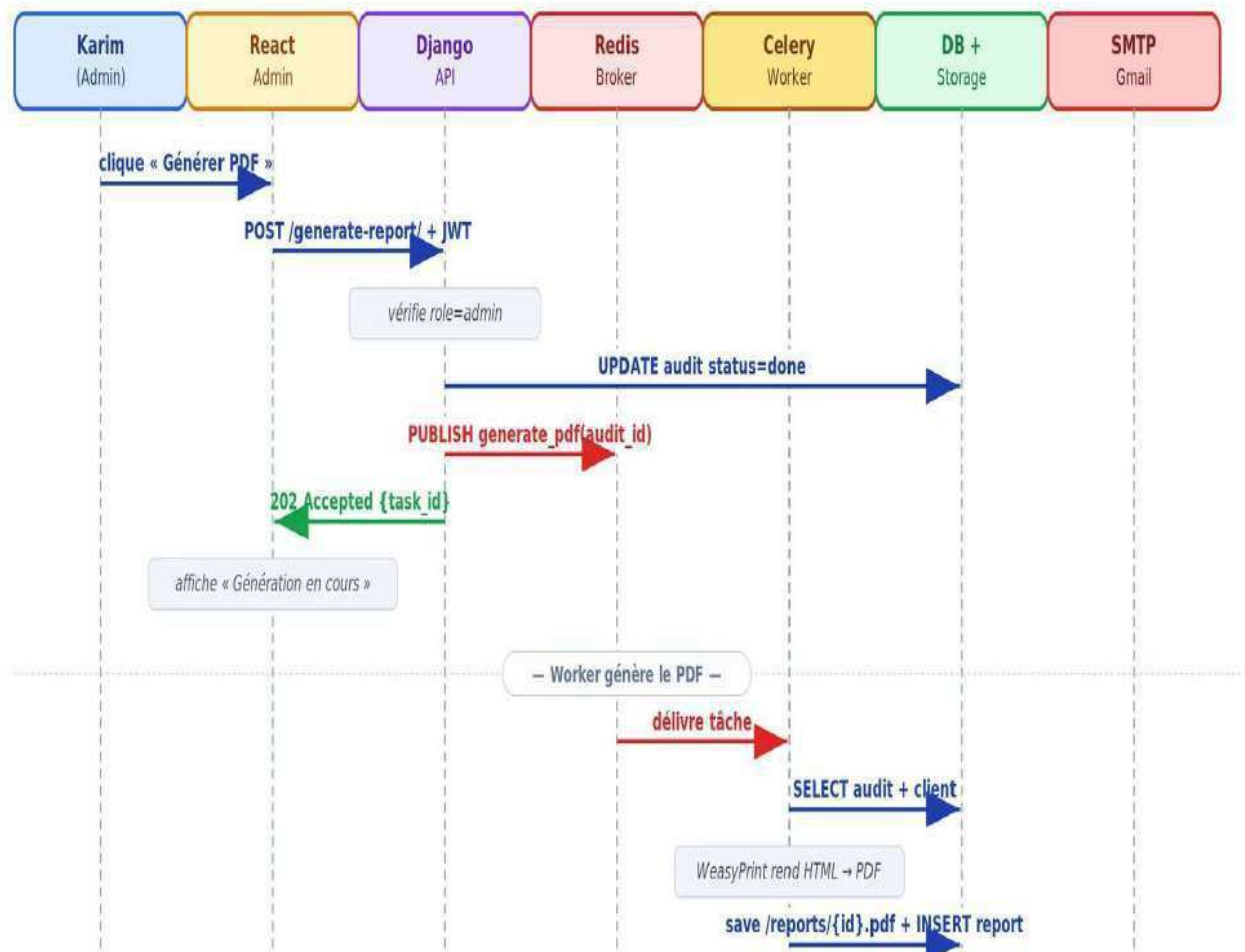


Figure 7 — Séquence Génération du rapport PDF

Messages échangés (en détail)

1. **Karim** → **AdminRequestDetail** : clique « Generate PDF report » avec les findings saisis.
2. **AdminRequestDetail** → **ApiClient** : POST /api/audits/{id}/generate-report/, body {findings, summary, security_score}.
3. **ApiClient** → **Django (PdfReportService)** : vérifie permission IsAdmin, vérifie status='done'.
4. **Django** → **PostgreSQL** : INSERT INTO reports_auditreport avec pdf_path=NULL (encore en cours).
5. **Django** → **Redis** : enfile generate_pdf_task.delay(report_id).
6. **Django** → **AdminRequestDetail** : 202 Accepted avec {report_id, status: "generating"}.
7. **Celery worker** ← **Redis** : dépile la tâche.
8. **Celery worker** → **Django templates** : rend report.html avec le contexte (findings, score, recommandations).
9. **Celery worker** → **WeasyPrint** : convertit HTML+CSS en PDF binaire.
10. **Celery worker** → **Storage** : sauvegarde dans /media/reports/CYB-2026-0042.pdf.
11. **Celery worker** → **PostgreSQL** : UPDATE auditreport SET pdf_path=....
12. **Celery worker** → **SMTP Gmail** : envoie e-mail « Rapport disponible » à Marie.
13. **Marie** → **ClientDashboard** : voit le lien « Download report » apparaître (polling 30 s).
14. **Marie** → **Django** : GET /api/audits/{id}/report/ avec JWT.
15. **Django** → **Marie** : URL signée 15 min, streaming du PDF.

Mermaid (rendu sur GitHub)

sequenceDiagram

```

actor Karim
participant Front as AdminRequestDetail
participant API as Django /audits
participant DB as PostgreSQL
participant Redis
participant Worker as Celery Worker
participant WP as WeasyPrint
participant Storage
participant SMTP
actor Marie

Karim->>Front: Clic Generate PDF report
Front->>API: POST /audits/{id}/generate-report/
API->>DB: INSERT auditreport (pdf_path=NULL)
API->>Redis: enqueue generate_pdf_task
API-->>Front: 202 Accepted
Worker->>Redis: dequeue
Worker->>Worker: render report.html
Worker->>WP: HTML+CSS to PDF
WP-->>Worker: PDF bytes
Worker->>Storage: save /media/reports/CYB-...pdf
Worker->>DB: UPDATE pdf_path
Worker->>SMTP: Email Rapport disponible
SMTP-->>Marie: Email reçu
Marie->>API: GET /audits/{id}/report/ + JWT
API-->>Marie: Stream PDF (signed URL 15min)

```

6. Documentation des APIs

6.1 APIs externes consommées

API externe	Usage	Justification
SMTP Gmail	Envoi des emails transactionnels (confirmation, changement statut, livraison rapport)	Gratuit jusqu'à 500 emails/jour, configuration simple, fiable. Migration possible vers SendGrid/Mailgun en V2.
Let's Encrypt	Certificats HTTPS automatiques	Gratuit, intégré nativement dans Vercel et Railway.

6.2 API interne : Endpoints REST

Convention : préfixe /api/, JSON pour requêtes et réponses, JWT dans Authorization: Bearer <token>, codes HTTP standards.

6.2.1 Authentification : /api/auth/

Méthode	URL	Auth	Description	Body	Réponse 2xx	Erreurs
POST	/api/auth/register/	—	Crée un compte client	{email, password, company_name, company_size, sector}	201 {user, access, refresh}	400, 409 email exists
POST	/api/auth/login/	—	Connexion + tokens JWT	{email, password}	200 {access, refresh, user}	401, 429 rate limit
POST	/api/auth/refresh/	—	Renouvelle access	{refresh}	200 {access}	401 refresh expired
POST	/api/auth/logout/	✓	Invalide le refresh	{refresh}	204	401
POST	/api/auth/password-reset/	—	Envoie email de reset	{email}	200 (toujours, anti-leak)	—

6.2.2 Demandes d'audit : /api/audits/

Méthode	URL	Auth	Rôle	Description
GET	/api/audits/	✓	client / admin	Liste paginée (client → ses demandes / admin → toutes)
POST	/api/audits/	✓	client	Crée une nouvelle demande
GET	/api/audits/{id}/	✓	owner / admin	Détail d'une demande
PATCH	/api/audits/{id}/	✓	admin	Modifie statut ou notes
DELETE	/api/audits/{id}/	✓	admin	Archive (soft delete)
POST	/api/audits/{id}/generate-report/	✓	admin	Lance la génération PDF (Celery)
GET	/api/audits/{id}/report/	✓	owner / admin	Télécharge le PDF (binaire)

Exemple : Création d'une demande

POST /api/audits/
 Authorization: Bearer eyJhbGc...
 Content-Type: application/json

```
{ "pack_id": 2, "scope_notes": "Audit du serveur de fichiers + 12 postes Windows."
}
```

HTTP/1.1 201 Created
 Content-Type: application/json

```
{
  "id": "8f2a...c9",
  "client": "user-uuid",
  "pack": {"id": 2, "name": "standard"},
  "status": "pending",
  "scope_notes": "Audit du serveur ...",
  "created_at": "2026-05-18T09:24:11Z"
}
```

6.2.3 Packs d'audit : /api/packs/

Méthode	URL	Auth	Description
GET	/api/packs/	—	Liste publique des 4 packs
GET	/api/packs/{id}/	—	Détail d'un pack

6.2.4 Formations : /api/training/

Méthode	URL	Auth	Description
GET	/api/training/modules/	✓	Liste des modules disponibles
GET	/api/training/modules/{id}/	✓	Contenu d'un module
POST	/api/training/modules/{id}/start/	✓	Marque comme commencé
POST	/api/training/modules/{id}/complete/	✓	Marque comme terminé

6.2.5 Profil utilisateur — /api/users/

Méthode	URL	Auth	Description
GET	/api/users/me/	✓	Profil de l'utilisateur courant
PATCH	/api/users/me/	✓	Modifie son propre profil
POST	/api/users/me/change-password/	✓	Change son mot de passe
DELETE	/api/users/me/	✓	Supprime son compte (RGPD)

6.3 Conventions transversales

- **Pagination** : ?page=1&page_size=20, réponse {count, next, previous, results}
- **Filtres** : ?status=pending&ordering=-created_at
- **Erreurs** : format unifié {error: "code", message: "Message FR", details: {...}}
- **Rate limiting** : 5 req/min sur /auth/login/ et /auth/password-reset/ (IP-based)
- **Versioning** : pas de v1 dans l'URL au MVP ; en V2 si breaking changes → /api/v2/

7. Stratégie SCM & QA

7.1 Source Code Management (Git)

Hébergeur : GitHub repo Tommy-JOUHANS/portfolio/cyberaudit (privé jusqu'à la soutenance, puis public).

Stratégie de branches (Git Flow allégé)

```
main          ← production uniquement (protégée, déploiement continu Vercel/
Railway)
develop       ← branche d'intégration (CI obligatoire)
feature/<nom> ← nouvelles fonctionnalités (1 PR = 1 feature)
fix/<nom>     ← corrections de bugs
hotfix/<nom>  ← correctifs urgents directement sur main
release/<v>   ← stabilisation avant release (à partir de W11)
```

Convention de commits — Conventional Commits

```
feat(audits): ajout du formulaire de demande
fix(auth): corrige le refresh token JWT expiré
docs(readme): met à jour la stack technique
test(audits): ajoute tests unitaires CRUD
chore(deps): bump Django 5.0.2 → 5.0.3
refactor(reports): extrait PDFGenerator en service
```

Règles de Pull Request

Règle	Détail
Revue obligatoire	Chaque PR validée par l'autre membre avant merge
CI verte	Tests + lint OK avant merge
Pas de push direct	main et develop protégées par GitHub Branch Protection
Description structurée	« Ce qui change / Pourquoi / Comment tester »
Squash merge	1 PR = 1 commit dans develop (historique propre)

7.2 Stratégie QA

Pyramide de tests

```

      /\
     /E2E\          5 % - Cypress (parcours critiques)
    /-----\
   / Intégr. \ 20 % - Tests intégration API + DB (Django + DRF)
  /-----\
 / Unitaires \ 75 % - Jest (front) + pytest (back)
/-----\

```

Outils retenus

Couche	Outil	Cible
Tests unitaires back	pytest + pytest-django	Modèles, services, serializers
Tests API back	pytest + APIClient (DRF)	Endpoints auth, audits, reports
Tests unitaires front	Jest + React Testing Library	Composants, hooks, utils
Tests E2E	Cypress	Inscription → demande → email (mock)
Tests sécurité	Postman (collection) + bandit	XSS, SQLi, brute-force, JWT tampering
Tests UX	3 sessions utilisateurs (W7-W10)	Persona Marie, parcours < 3 min
Coverage	coverage.py + jest --coverage	Cible : 80 % min apps critiques
Lint	ruff + black ; eslint + prettier	Pre-commit hook
Type checking	mypy ; propTypes ou TS (V2)	À partir de W4

7.3 Tests sécurité manuels (W7-W10)

Test	Méthode	Résultat attendu
XSS persistant	<script>alert(1)</script> dans scope_notes	Échappé / sanitized
SQL injection	' OR 1=1 -- dans email login	Aucune fuite, ORM bind
Brute-force	100 tentatives login	Bloqué dès la 6 ^e (rate-limit)
JWT tampering	Modifier role dans le payload	Signature invalide → 401
CSRF	POST sans header Authorization	401
HSTS / CSP	Headers HTTP en prod	Présents et stricts

7.4 Pipeline CI/CD

GitHub Actions

```
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]

jobs:
  backend:
    runs-on: ubuntu-latest
    steps:
      - checkout
      - setup-python 3.11
      - install: pip install -r backend/requirements-dev.txt
      - lint: ruff check . && black --check .
      - test: pytest --cov=apps --cov-fail-under=80

  frontend:
    runs-on: ubuntu-latest
    steps:
      - checkout
      - setup-node 18
      - install: npm ci
      - lint: npm run lint
      - test: npm test -- --coverage
      - build: npm run build
```

Déploiement continu

Environnement	Déclencheur	Cible	URL
Preview	Toute PR	Vercel preview + Railway branch deploy	pr- XX.preview.cyberaudit.app
Staging	Merge sur develop	Vercel + Railway staging	staging.cyberaudit.app
Production	Merge sur main (release tag)	Vercel + Railway prod	cyberaudit.app

Stratégie de release

Versioning SemVer : 0.1.0 (W4 MVP partiel) → 0.5.0 (W6 async) → 1.0.0 (W12
soutenance)

- Tag git annoté à chaque release : `git tag -a v1.0.0 -m "MVP soutenance Holberton"`
- Changelog tenu à jour dans CHANGELOG.md

Annexes

8.1 Glossaire

Terme	Définition
MVP	Produit Minimum Viable (MVP) : version minimale livrable du produit
JWT	Jeton Web : norme d'authentification sans état
MoSCoW	Méthode de priorisation : Obligatoire / Recommandé / Possible / Non requis
CRUD	CRUD : Créer / Lire / Mettre à jour / Supprimer
ORM	ORM : Abstraction de base de données
CI/CD	CI/CD : Intégration continue / Déploiement continu
GDPR	RGPD : Règlement général sur la protection des données
HSTS HTTP	Strict Transport Security : en-tête anti-rétrogradation
CSP	Content Security Policy : en-tête anti-XSS
SemVer	Versionnement sémantique : MAJEURE.MINEURE.CORRECTIF
SPA	Application monopage (SPA)

8.2 Cartographie du Niveau 3 ↔ Compétences RNCP 5

Référence RNCP 38038 : Développeur Web et Web Mobile (Titre professionnel délivré par le Ministère du Travail). Ce document couvre les blocs suivants :

Bloc RNCP	Compétence	Section Documentation
BC01 "Développer la partie front-end"	Maquetter une application	§2.3 (mockups)
BC01	Réaliser des interfaces statiques et dynamique	§4.3 (Composants React)
BC01	Développer la partie frontend	§3 (archi), §6 (API)
BC02 "Développer la partie back-end"	Créer une base de données	§4.1 (ERD)
BC02	Développer les composants d'accès aux données	§4.2 (modèles Django)
BC02	Développer la partie backend	§6 (API REST)
BC02	Élaborer et mettre en œuvre des composants	§4.2 (services), §5 (séquences)
Transversale	Travailler en équipe agile / Versioning	§7 (SCM/QA)

8.3 Sources et références techniques

Reference	Links
Django REST Framework : Documentation	https://www.django-rest-framework.org
SimpleJWT : JWT for Django	https://django-rest-framework-simplejwt.readthedocs.io
Celery : Distributed Task Queue	https://docs.celeryq.dev
WeasyPrint : HTML to PDF	https://weasyprint.org
OWASP Top 10 :security repository	https://owasp.org/www-project-top-ten
Conventional Commits	https://www.conventionalcommits.org
RNCP 38038 : Web and Mobile Web Developer	https://www.francecompetences.fr/recherche/rncp/38038
ANSSI : Overview of the cyber threat 2023	https://www.ssi.gouv.fr

8.4 Liens internes au sein du projet

- Stage 1: Project Charter (FR)
- Stage 2: Planning & Gantt (FR)
- Figma Wireframes
- MVP Code Repo

Document préparé dans le cadre du Stage 3 : Documentation Technique, Holberton School Dijon, promotion 2026. Conçu pour être réutilisé tel quel dans le dossier de certification RNCP 5 « Développeur Web et Web Mobile ».